

An Archetypal Analysis of Archetypal Analysis

732A76 Research Project

Thalgamuwe Gedara Ashen Akalanka Weligalle

December 10, 2025

Abstract

Archetypal Analysis (AA) was introduced by Cutler and Breiman in 1994 [1]. It is a method designed to extract distinct structural patterns, called archetypes, from observational data, where each data point is approximated as a convex mixture of these archetypes [2]. Although AA has been successfully applied in several domains, its potential as a topic modeling framework for scientific literature remains relatively underexplored. In this project, we investigate the use of Archetypal Analysis as a topic modeling approach for analyzing research abstracts related to AA itself, thereby creating a self-referential analysis of the literature.

The experimental results of this research demonstrate that AA produces highly interpretable and sparse topic representations that capture extremal semantic structures within the literature, often more distinctly than probabilistic or centroid-based methods. This work highlights the suitability of Archetypal Analysis as a geometric framework for topic modeling and provides insights into the structure and evolution of research trends within the AA research community.

1 Introduction

Archetypal Analysis (AA), introduced by Cutler and Breiman [1], provides the theoretical foundation for this project. According to their work, the original motivation for Archetypal Analysis arose from a practical problem in 1988, when the Swiss Army needed to design improved protective face masks for soldiers. This task was assigned to Flury and Riedwyl, who analyzed a large dataset of human head measurements and observed interesting geometric patterns in the data.[1]

A natural question that emerged from this study was whether a small number of “pure” head types, called *archetypes*, could explain the observed variability in the dataset. In other words, could the roughly 200 measured head shapes be represented as convex mixtures of a few extreme, idealized head shapes? To address this problem, Cutler and Breiman developed Archetypal Analysis.

In this work, we build on the theoretical framework of Archetypal Analysis and explore its application as a topic modeling method for scientific text data.

1.1 Archetypal Analysis

Let’s consider a dataset represented by a matrix $\mathbf{X} \in \mathbb{R}^{N \times M}$, where N is the number of data samples and M is the number of features. The dataset can be written as

$$\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\},$$

where each $\mathbf{x}_i \in \mathbb{R}^M$ represents one observation.

The goal of Archetypal Analysis is to approximate the data matrix $\mathbf{X}$ by a low-rank decomposition

$$\mathbf{X} \approx \mathbf{S}\mathbf{A},$$

where $\mathbf{S} \in \mathbb{R}^{N \times K}$ and $\mathbf{A} \in \mathbb{R}^{K \times M}$.

Here, $\mathbf{S}$ is the coefficient (or weight) matrix that expresses each data point as a convex combination of archetypes, and $\mathbf{A}$ is the matrix containing the K archetypes themselves. [2] The common data representation in Archetypal Analysis expresses each data point $\mathbf{x}_i$ as a convex combination of K archetypes $\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_K$:

$$\mathbf{x}_i \approx \mathbf{A}^\top \mathbf{s}_i,$$

where $s_i \geq 0$ and $\sum_{j=1}^K s_{ij} = 1$. The vector s_i contains the weights of the convex combination, and these vectors are stacked row-wise to form the matrix $S \in \mathbb{R}^{N \times K}$. The matrix $A \in \mathbb{R}^{K \times M}$ contains the archetypes $a_1, a_2, \dots, a_K$ as its rows and is referred to as the design matrix of archetypes.

Each archetype can itself be expressed as a convex combination of the original data points:

$$a_k = X^\top c_k,$$

where $c_k \geq 0$ and $\sum_{i=1}^N c_{ki} = 1$. These coefficient vectors are stacked row-wise to form the matrix

$$C \in \mathbb{R}^{K \times N}.$$

From these definitions, it follows that

$$A = CX.$$

Due to the convexity constraints, both coefficient matrices S and C are row-stochastic, meaning that all entries are non-negative and each row sums to one.

1.2 Residual Sum of Squares (RSS)

The weight matrices are obtained by minimizing the Residual Sum of Squares (RSS) as the objective function, defined as

$$\text{RSS} = \|X - SCX\|_F^2 = \|X - SA\|_F^2.$$

Here, $\|\cdot\|_F^2$ denotes the squared Frobenius norm.

Optimizing the RSS in order to estimate the optimal weight matrices S and C leads to a non-convex optimization problem. However, the problem is convex in S when C is fixed, and convex in C when S is fixed [2].

For readers interested in the optimization procedure, the pseudocode is presented in Subsection 6.1. Further technical details can be found in the references [2], [3].

1.3 Research Aim and Targets

The goal of this project is to apply Archetypal Analysis (AA) as a topic modeling method for textual data, specifically focusing on scientific abstracts related to Archetypal Analysis. The objective is to use AA to analyze the structure and themes of the AA research literature.

Baseline methods for comparison include Non-negative Matrix Factorization (NMF), K-means clustering, and Latent Dirichlet Allocation (LDA). Furthermore, this project aims to validate and reproduce findings reported in previous studies [3].

2 Methodology

What did you do? Adjust the section title.

3 Experiments

Experimental details.

Data. Asd

Baselines. Sdf

Etc. qwerty

4 Discussion

How to interpret the results you obtained?

5 Conclusion

Very brief summary and conclusion of your work. [2]

References

- [1] A. Cutler and L. Breiman, “Archetypal analysis,” *Technometrics*, vol. 36, no. 4, pp. 338–347, 1994.
- [2] A. Alcacer, I. Epifanio, S. Mair, and M. Mørup, “A survey on archetypal analysis,” *arXiv preprint arXiv:2504.12392*, 2025.
- [3] M. Mørup and L. K. Hansen, “Archetypal analysis for machine learning and data mining,” *Neurocomputing*, vol. 80, pp. 54–63, 2012.

6 Appendix

6.1 AA model

```
import numpy as np

def project_to_simplex(v):
    """
    Project a vector v onto the probability simplex:
    {w | w >= 0, sum(w) = 1}.
    """
    v = np.asarray(v, dtype=float)
    n = v.shape[0]
    u = np.sort(v)[::-1]
    cssv = np.cumsum(u)
    rho = np.nonzero(u * np.arange(1, n + 1) > (cssv - 1))[0][-1]
    theta = (cssv[rho] - 1.0) / (rho + 1.0)
    w = np.maximum(v - theta, 0.0)
    return w

def solve_simplex_least_squares(B, y, max_iter=500, tol=1e-6):
    """
    Solve: min_s || B^T s - y ||^2 subject to s >= 0, sum(s) = 1
    using projected gradient descent (PGD).

    Note: B should be passed as the matrix (not transposed), and this
    function will compute B^T @ (B^T @ s - y) for the gradient.
    """
    K = B.shape[0] # B is (M, K) so we want K archetypes
    s = np.ones(K) / K # start uniform

    # For min ||B^T s - y||^2, gradient is: B @ (B^T @ s - y)
    # Step size based on Lipschitz constant
    step_size = 1.0 / (np.linalg.norm(B, 2) ** 2 + 1e-12)

    prev_val = np.inf
    for _ in range(max_iter):
        grad = B @ (B.T @ s - y) # Gradient for ||B^T s - y||^2
        s = s - step_size * grad
        s = project_to_simplex(s)

        val = 0.5 * np.linalg.norm(B.T @ s - y) ** 2
        if abs(prev_val - val) < tol * (1 + prev_val):
            break
        prev_val = val
    return s

def archetypal_analysis(X, K, max_iter=50, tol=1e-5, random_state=0,
    verbose=True):
    """
    Archetypal Analysis (AA) via alternating optimization.

    Follows Algorithm 1: min ||X - S*C*X||_F^2 = min ||X - S*A||_F^2
    """
```

where $A = C \cdot X$ are the archetypes.

Args:

X: (N, M) data matrix (N samples, M features)

K: number of archetypes

max_iter: maximum iterations

tol: convergence tolerance

random_state: random seed

Returns:

S: (N, K) coefficients for data points in archetype space ($s_n \geq 0$, $\sum=1$)

C: (K, N) coefficients for archetypes in data space ($c_k \geq 0$, $\sum=1$)

A: (K, M) archetypes where $A = C \cdot X$

"""

rng = np.random.default_rng(random_state)

N, M = X.shape

--- Initialize A with random rows of X ---

idx = rng.choice(N, size=K, replace=False)

A = X[idx, :].copy()

S = np.ones((N, K)) / K

C = np.ones((K, N)) / N

prev_obj = np.inf

for iteration in range(max_iter):

--- Step 1: Update S row by row ---

For each data point n, solve: $\min ||A^T s_n - x_n||^2$ s.t. $s_n \geq 0$, $\sum(s_n) = 1$

for n in range(N):

S[n, :] = solve_simplex_least_squares(A, X[n, :])

--- Step 2: Update A via least squares ---

$A = (S^T S)^{-1} S^T X$

StS = S.T @ S + 1e-10 * np.eye(K) # Add small regularization for stability

A = np.linalg.solve(StS, S.T @ X)

--- Step 3: Update C row by row ---

For each archetype k, solve: $\min ||X^T c_k - a_k||^2$ s.t. $c_k \geq 0$, $\sum(c_k) = 1$

for k in range(K):

C[k, :] = solve_simplex_least_squares(X, A[k, :])

--- Step 4: Update A using C ---

$A = C \cdot X$ (archetypes are convex combinations of data points)

A = C @ X

--- Check convergence ---

$RSS = ||X - S \cdot A||_F^2 = ||X - S \cdot C \cdot X||_F^2$

obj = 0.5 * np.linalg.norm(X - S @ A, "fro") ** 2

if verbose and iteration % 5 == 0:

print(f"Iteration {iteration:3d}, RSS = {obj:.4e}")

if abs(prev_obj - obj) < tol * (1 + prev_obj):

if verbose:

```

        print(f"Converged at iteration {iteration}, RSS = {obj:.4e}
              ")
    break
    prev_obj = obj

return S, C, A

```

Code template: Your report should include the main code in the appendix section. To make your code readable in the PDF file, you can use a code-style template by adding the lines

```

\usepackage{xcolor}
\definecolor{codegreen}{rgb}{0,0.6,0}
\definecolor{codegray}{rgb}{0.5,0.5,0.5}
\definecolor{codepurple}{rgb}{0.58,0,0.82}
\definecolor{backcolour}{rgb}{0.95,0.95,0.92}
\usepackage{listings}
\lstdefinestyle{mystyle}{
    backgroundcolor=\color{backcolour},
    commentstyle=\color{codegreen},
    keywordstyle=\color{magenta},
    numberstyle=\tiny\color{codegray},
    stringstyle=\color{codepurple},
    basicstyle=\footnotesize\ttfamily,
    breakatwhitespace=false,
    breaklines=true, captionpos=b,
    keepspaces=true, numbers=left,
    numbersep=5pt, showspaces=false,
    showstringspaces=false,
    showtabs=false, tabsize=2}
\lstset{style=mystyle}

```

before `\begin{document}` and use `\lstinputlisting[language=python]{code.py}` to insert code to the appendix section.

Another choice: If you write your code using jupyter notebook, you can also directly export the notebook to a PDF and concatenate it to the report as an additional appendix.

7 Style

This section was copied from advice for writing a masters thesis but is considered also helpful for this report.

Writing

1. **Write in English.** Either UK or US English is fine, but try to be consistent.
2. **Write correct English.** Your editor most likely supports spell correction. Use it!
3. **Use active voice.** “We trained the neural network” is preferable over “The neural network was trained”.
4. **Use consistent terminology.** Many concepts are known under different names (e.g. artificial neural networks, neural networks, deep neural networks, deep learning), but to avoid confusion you should pick one and consistently use only that.

5. **Avoid abbreviations.** As a rule of thumb, only ever consider abbreviations that are either (i) generally used and understood by everyone in the field, e.g., PDE for Partial Differential Equation, or (ii) used everywhere in your report, e.g., at least three times in the same section. Consider adding a list of abbreviations/acronyms in the beginning.
6. **Use correct references.** When referring to a specific algorithm, equation, figure, section, or table, the (usually lowercase) object gets capitalized, e.g., “as we can see in Figure 5” vs “as depicted in the figure”. Use `\eqref` when referencing equations.
7. **Only cite academic work.** If you need a statement from a company website or blog, set a footnote. Only cite proper academic output. Some software packages can be seen as academic work. If you are unsure what to cite, cite the paper that introduced the idea/concept and an article or book describing it best. Do not cite random papers for well-known concepts!
8. **Use a consistent citation style.** The Harvard style is preferred. Note that there is a difference between an active citation (you would read it out loud) and a passive one (you would not read it out loud). Two examples: Author et al. (2023) show a result. A new result was shown (Author et al., 2023).

Mathematics

Mathematical writing is fundamentally different from creative and expository writing. In addition to the advice below, please refer to the “Ten Simple Rules for Mathematical Writing” by Dimitri P. Bertsekas¹.

1. **Use L^AT_EX.** Try to avoid writing in MS Word whenever possible.
2. **Never start a sentence with a mathematical symbol.** Never.
3. **Respect conventions.** Although you are technically free to call your inputs y and your outputs x , the reader will dislike you for it.
4. **Math is always a part of a sentence.** There is no equation between two sentences. An equation is a part of a sentence and thus needs proper punctuation marks!
5. **Use single-letter variables.** If necessary, use subscripts for additional clarity. It is also advised to specify the space of newly introduced variables, e.g., $\lambda \geq 0$ (or $\lambda \in \mathbb{R}_{\geq 0}$, or $\lambda \in \mathbb{R}_0^+$). In addition, be consistent. The meaning of a variable should not change during sections.

Presentation

1. **Start with the big picture.** Start most sections and subsections by reminding the reader about the big picture, where you are, and what you’ll describe next.
2. **Present things in logical, not chronological, order.** The point of the report is to communicate your findings in the way that makes the most sense for the *reader*. This is often not the same way that you arrived at them. See the note on “How to write a scientific report”² by Michael Thuné for a longer explanation of this point.
3. **Structure your text.** Use sections, subsections, subsubsections, and paragraphs wisely. Not every subsection needs a number, but headings help to structure the text. Thus, if you have two pages of full text, it might be wise to break the text and introduce a heading somewhere.

¹https://www.mit.edu/~dimitrib/Ten_Rules.html

²<https://www.it.uu.se/edu/course/homepage/projektTDB/vt05/examination/rapport.pdf>

4. **Table of contents.** If your table of contents is longer than 1.5-2 pages, reduce. The reduction should not be done by removing structure but rather by not enlisting every subsubsection in the table of contents. Not every subsection needs a number!
5. **State your contributions.** It should be clearly stated what your contributions are. If two students jointly write the thesis, it should be clear which student contributed what. Ideally, this is done on a chapter basis or within an *outline* paragraph at the end of the introduction.
6. **Use tables sparingly.** Use booktabs and read <https://nhigham.com/2019/11/19/better-latex-tables-with-booktabs/>. Table captions go above tables while figure captions are placed below figures.
7. **Use images.** Images often help to illustrate concepts and ease understanding. State the image sources if necessary and use appropriate image qualities.
8. **Do proper figures.** Figures should have clearly defined axis labels and include a legend. Any text in the figure should approximately match the font size of the thesis main text, thus, it should be readable without zooming in. A transparent grid in the background usually improves the appearance and readability of the figure. Try to use colors that also work for colorblind readers and look good when the thesis is printed in black and white. Images, e.g., created with `matplotlib`, are best saved in pdf format.